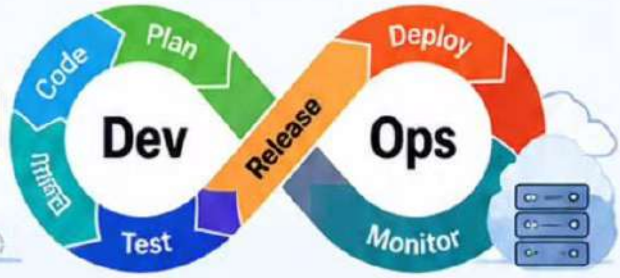
PAGE 1

DevOps Top Interview Questions



DevOps Fundamentals (1–10)

1. What is DevOps?

DevOps is a combination of **Development (Dev)** and **Operations (Ops)**. It aims to automate and integrate the processes between software development and IT operations to deliver applications faster and more reliably.

2. What are the key principles of DevOps?

- Collaboration & Communication
- Automation
- Continuous Improvement
- Customer Focus
- Transparency & Feedback
- Shared Responsibility

3. Explain the DevOps lifecycle.

Plan → Code → Build → Test
Release → Deploy → Monitor

4. What is Continuous Integration (CI)?

CI is the practice of **automatically integrating** code changes from multiple developers into a shared repository frequently, often several times a day.

5. What is Continuous Delivery (CD)?

CD ensures that code changes are automatically tested and **prepared for release** to production, but the final deployment may require manual approval.

6. What is Continuous Deployment?

It is an extension of CD where every change that passes all tests is **automatically deployed** to production without manual intervention.

7. What is Infrastructure as Code (IaC)?

IaC is the practice of **managing and provisioning infrastructure through code** instead of manual configuration.

8. What are the benefits of DevOps?

- Faster Delivery
- Improved Collaboration
- Reduced Errors
- Cost Efficiency
- High Quality
- Better Scalability

9. What is Agile, and how is it related to DevOps?

Agile is a software development methodology focused on **iterative and incremental delivery**. DevOps adopts Agile principles to speed up development and delivery.

10. What is the difference between DevOps and Agile?

- **Agile** is about how we develop (process & methodology).
- **DevOps** is about how we **deliver** and **operate** (culture & tools).

Key Takeaway

PYTHON LIFE

DevOps is all about collaboration, automation, and continuous improvement to deliver high-quality software faster and more efficiently.

Pro Tip: Focus on real-world examples and concepts during your interview!

PAGE 2

DevOps Top Interview Questions

GIT, JENKINS & CI/CD (11–20)

These questions cover version control, automation, and the CI/CD pipeline which are essential for any DevOps engineer.

11 What is Git, and why is it used?

- Git is a distributed version control system.
- It is used to track changes in source code, enable collaboration, maintain history, and support branching and merging.

12 Explain Git branching strategies.

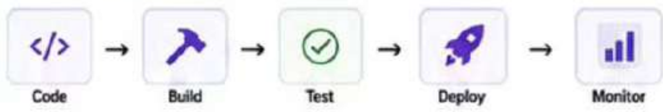
- Common strategies: Git Flow, GitHub Flow, GitLab Flow.
- Helps in managing features, releases, and hotfixes in a structured way.

13 What is a merge conflict, and how do you resolve it?

- A merge conflict occurs when Git cannot automatically merge changes.
- Resolve by: opening the conflicted file, editing manually, staging the file, and committing.

14 What is Jenkins?

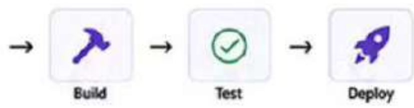
- Jenkins is an open-source automation server.
- It is used to automate building, testing, and deploying applications as part of CI/CD.



15 What is a Jenkins Pipeline?

- A Jenkins Pipeline is a suite of plugins that supports implementing and integrating continuous delivery pipelines into Jenkins.
- Defined as code (Jenkinsfile).

```
pipeline {
  agent any
  stages {
    stage('Build') { steps { ... } }
    stage('Test')  { steps { ... } }
    stage('Deploy) { steps { ... } }
  }
}
```



16 Difference between Declarative and Scripted Pipelines.

- Declarative: Structured, easier syntax, Jenkinsfile with predefined blocks.
- Scripted: More flexible, uses Groovy scripting.

Declarative Pipeline

- Easier to read and write
- Follows strict structure
- Easier syntax
- Suitable for standard pipelines

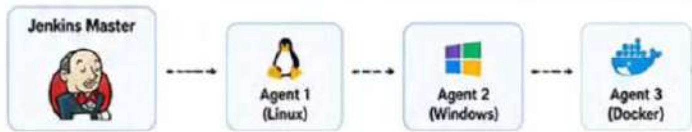
VS

Scripted Pipeline

- More flexible
- Uses Groovy code
- Complex logic possible
- Suitable for complex pipelines

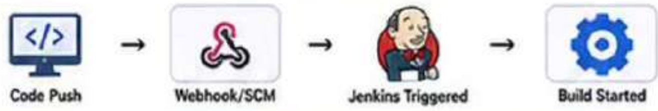
17 What are Jenkins agents (nodes)?

- Agents (nodes) are machines that execute Jenkins jobs.
- They can be any machine (Linux, Windows, Docker) configured to connect with Jenkins master.



18 How do you trigger Jenkins builds automatically?

- Using Webhooks, SCM polling, cron jobs, or upstream jobs.
- Most common: Webhook (on code push) and SCM Polling.



19 Explain a typical CI/CD pipeline.

- Code Commit → Build → Test → Security Scan → Deploy to Staging → Approve → Deploy to Production → Monitor

20 What are pipeline stages?

- Stages are distinct phases in a pipeline (Build, Test, Deploy).
- They help in organizing the pipeline into logical steps.

Example Pipeline Stages

Checkout → Build → Test → Deploy → Post Actions

Pro Tip: Practice creating pipelines and automating builds – Hands-on experience makes a huge difference in interviews!

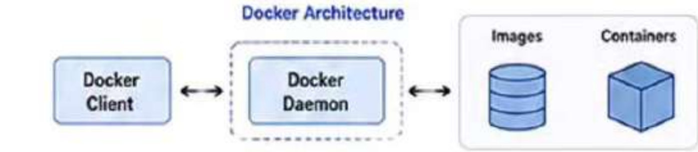
DevOps Top Interview Questions

DOCKER & KUBERNETES (21–30)

These questions focus on containerization with Docker and orchestration with Kubernetes.

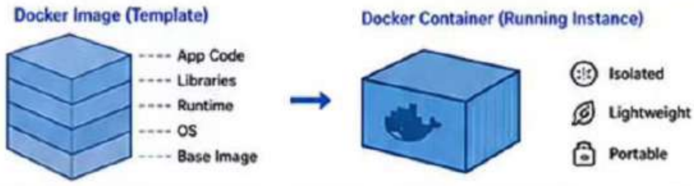
21 What is Docker?

- Docker is a platform for developing, shipping, and running applications inside containers.
- It enables packaging an application with all its dependencies into a portable unit.



22 What is the difference between Docker Image and Container?

- **Image:** A read-only template that contains application code, runtime, libraries, and dependencies.
- **Container:** A running instance of an image.

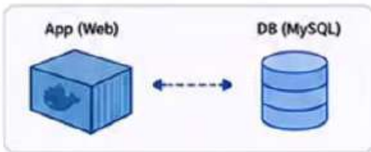


23 What is Docker Compose?

- Docker Compose is a tool for defining and running multi-container Docker applications.
- It uses a YAML file to configure application services and their dependencies.

docker-compose.yml

```yaml
version: '3'
services:
  web:
    image: nginx
    ports:
      - "80:80"
  db:
    image: mysql
    environment:
      - MYSQL_ROOT_PASSWORD=root
```

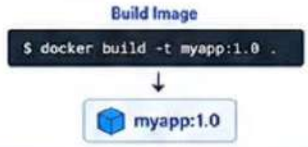


24 Explain Dockerfile.

- A Dockerfile is a script containing instructions to build a Docker image.
- It defines the base image, dependencies, commands, and configuration.

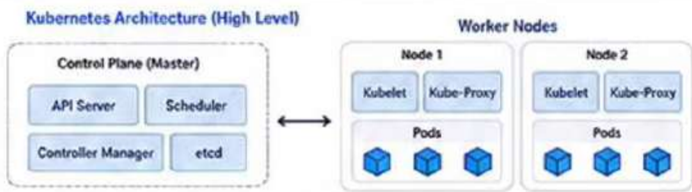
Dockerfile Example

```dockerfile
FROM nginx:alpine
COPY . /usr/share/nginx/html
EXPOSE 80
CMD ["nginx", "-g", "daemon off;"]
```



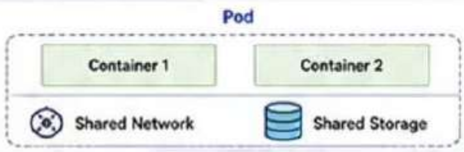
25 What is Kubernetes?

- Kubernetes (K8s) is an open-source platform for automating deployment, scaling, and management of containerized applications.
- It provides self-healing, load balancing, service discovery, and more.



26 What is a Pod in Kubernetes?

- A Pod is the smallest deployable unit in Kubernetes.
- It can contain one or more containers that share network, storage, and configuration.



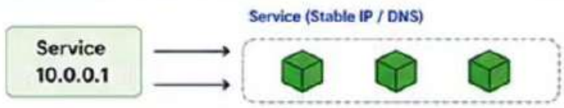
27 Difference between Deployment and StatefulSet.

- Deployment is used for stateless applications.
- StatefulSet is used for stateful applications that require stable network identity and persistent storage.

	Deployment	StatefulSet
Use Case	Stateless applications	Stateful applications
Pod Identity	Pods are interchangeable	Stable, unique identity
Storage	Ephemeral	Persistent
Scaling	Easy (Replica based)	Ordered & Controlled
Example	Web apps, APIs	Databases, Kafka, Zookeeper

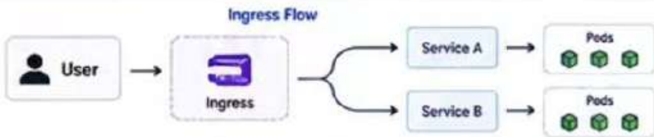
28 What is a Service in Kubernetes?

- A Service is an abstraction which defines a logical set of Pods and a policy to access them.
- It provides stable IP and DNS name for accessing Pods.



29 What is Ingress?

- Ingress is an API object that manages external access to the services in a cluster, typically HTTP/HTTPS.
- It provides load balancing, SSL termination, and routing.



30 How does Kubernetes perform rolling updates?

- Kubernetes updates Pods gradually with zero downtime.
- It creates new Pods with updated version and terminates old Pods after they are ready.

Pro Tip: Hands-on practice with Docker and Kubernetes is the key to cracking real-world DevOps interviews!

DevOps Top Interview Questions

TERRAFORM, CLOUD & MONITORING (31–40)

Terraform | AWS | Monitoring

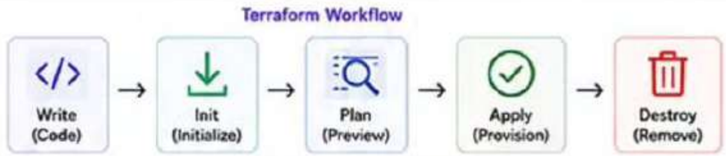
These questions cover Infrastructure as Code with Terraform, key cloud concepts (AWS) and monitoring tools.

31 What is Terraform?

- Terraform is an Infrastructure as Code (IaC) tool that allows you to define and provision infrastructure using declarative configuration files.
- It supports multiple cloud providers.

32 Explain the Terraform workflow.

- The standard workflow is: Write (Code) → Init → Plan → Apply → Destroy.

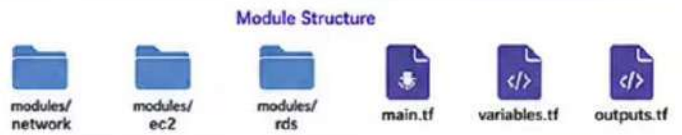


33 What is a Terraform state file?

- The state file tracks the current state of your infrastructure.
- It maps real-world resources to your Terraform configuration.

34 What are Terraform modules?

- Modules are reusable containers for Terraform code.
- They help in organizing code, reusing components, and maintaining best practices.

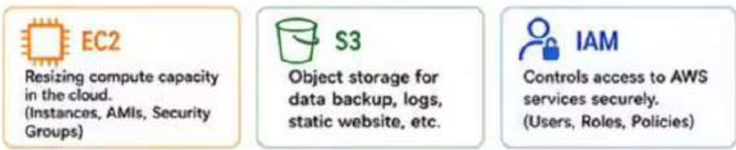


35 What is Infrastructure as Code?

- IaC is the practice of managing and provisioning infrastructure through code instead of manual processes.
- It enables automation, version control, and consistency.

36 Explain AWS EC2, S3, and IAM.

- EC2: Virtual servers in the cloud.
- S3: Scalable object storage service.
- IAM: Manages users, groups, roles and permissions.

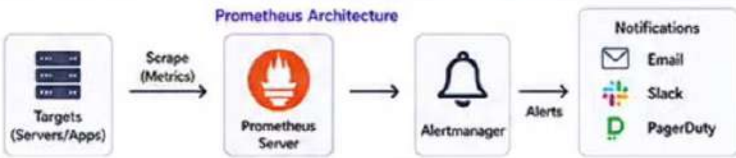


37 What is a Virtual Private Cloud (VPC)?

- VPC is a logically isolated network in AWS where you can launch AWS resources.
- You can define IP ranges, subnets, route tables, gateways, and security settings.

38 What is Prometheus?

- Prometheus is an open-source monitoring and alerting toolkit.
- It collects metrics, stores time-series data, and provides powerful querying with PromQL.

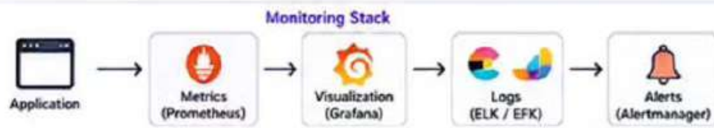


39 What is Grafana?

- Grafana is an open-source platform for monitoring and observability.
- It visualizes metrics from Prometheus, InfluxDB, MySQL and more with beautiful dashboards.

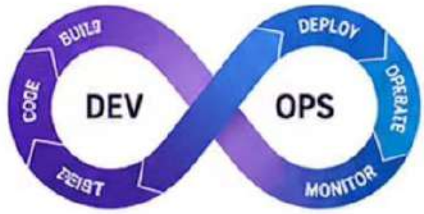
40 How do you monitor applications in production?

- Collect metrics, logs, and traces.
- Use tools like Prometheus, Grafana, ELK/EFK Stack.
- Set up alerts, dashboards and incident notifications.



Pro Tip: Understand the core concepts deeply and practice hands-on with Terraform, AWS and Monitoring tools.

DevOps Top Interview Questions

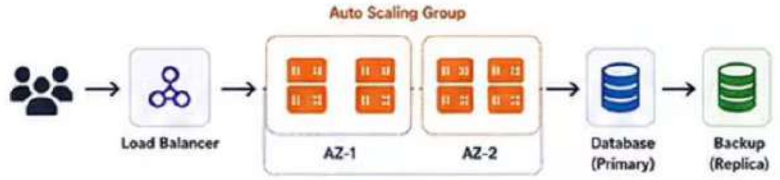


ADVANCED DEVOPS & SCENARIO QUESTIONS (41–50)

These scenario-based questions test your practical knowledge, problem-solving skills, and real-world DevOps experience.

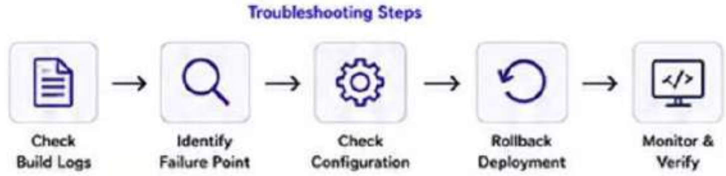
41 How would you deploy a highly available application?

- Use multi-AZ/region deployment.
- Load balancer to distribute traffic.
- Auto Scaling for handling load.
- Use health checks and monitoring.
- Database with replication & backups.



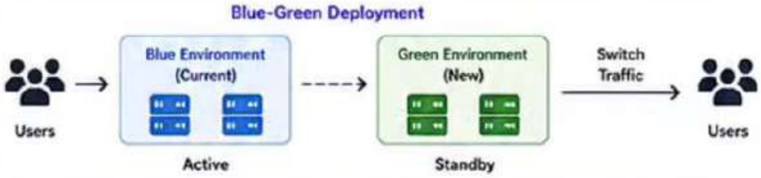
42 How do you troubleshoot a failed deployment?

- Check pipeline logs and error messages.
- Verify recent code changes/commit.
- Check environment & configuration.
- Rollback to last stable version.
- Monitor application & infrastructure.



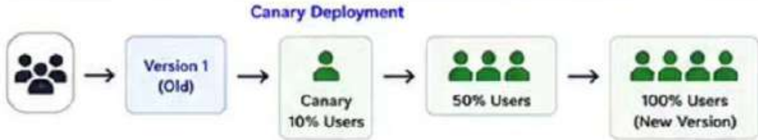
43 Explain Blue-Green Deployment.

- Two identical environments: Blue (current) and Green (new).
- Switch traffic to Green after successful testing.
- Easy rollback by switching back to Blue.



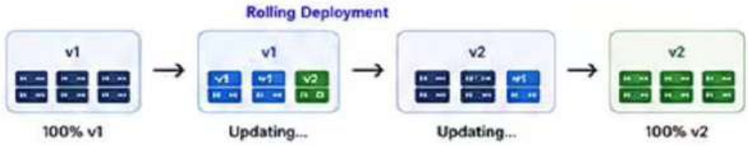
44 Explain Canary Deployment.

- Release to a small subset of users first.
- Monitor the application.
- Gradually roll out to all users.
- Reduces risk of full-scale failure.



45 What is Rolling Deployment?

- Gradually replace old version instances with new ones.
- Ensures zero downtime.
- If issue occurs, stop rollout and rollback.



46 How do you secure a CI/CD pipeline?

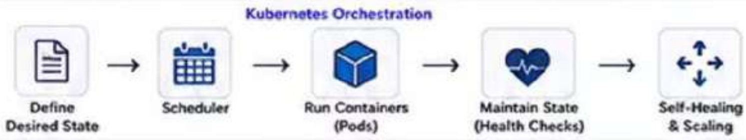
- Use least privilege access.
- Store secrets in Vault/Secrets Manager.
- Scan code and dependencies for vulnerabilities.
- Use signed artifacts and secure agents.
- Audit logs and monitor pipeline activities.

47 What are secrets management tools?

- Tools used to store and manage sensitive data securely.
- Examples: HashiCorp Vault, AWS Secrets Manager, Azure Key Vault.

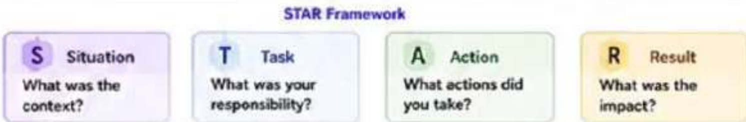
48 Explain container orchestration.

- Automates deployment, scaling, networking, and management of containers.
- Kubernetes is the most popular orchestration tool.



49 Describe a DevOps project you worked on.

- Use STAR method: Situation, Task, Action, Result.
- Highlight tools, your role, challenges, and impact.
- Example: CI/CD pipeline, cloud migration, monitoring setup.



50 Why should we hire you for a DevOps role?

- Show your technical skills, tools knowledge.
- Highlight problem-solving, automation mindset.
- Emphasize teamwork, ownership, and continuous learning.

Pro Tip: Practice real-world scenarios, stay updated with tools, and always think about automation, scalability and reliability!